

PLAID-PRF: Pseudo-Relevance Feedback with Centroid-like Tokens in PLAID

Xiao Wang
University of International Business
and Economics
Beijing, China
xiao.wang@uibe.edu.cn

Sean MacAvaney
University of Glasgow
Glasgow, United Kingdom
sean.macavaney@glasgow.ac.uk

Craig Macdonald
University of Glasgow
Glasgow, United Kingdom
craig.macdonald@glasgow.ac.uk

Abstract

Multi-vector dense retrieval models, such as ColBERT, achieve strong retrieval effectiveness by modelling fine-grained token-level interactions between queries and documents. Methods such as PLAID use centroid-based quantisation of each token’s vector to reduce the index size and speed up retrieval while maintaining strong effectiveness. In this work, we introduce PLAID-PRF, a method that performs Pseudo-Relevance Feedback (PRF) over PLAID to reformulate ColBERT’s query vectors based on the top-retrieved results. In contrast with prior methods that perform PRF on multi-vector retrieval models, PLAID-PRF keeps computational costs low by leveraging the internal PLAID centroid vectors, treating them similarly to tokens in traditional PRF methods. The method selects a small and diverse set of high-utility expansion vectors and appends them to the original query, rerunning PLAID to refine both candidate generation and final scoring. Extensive experiments on the standard in-domain MSMARCO and four out-of-domain BEIR benchmarks show that PLAID-PRF consistently improves retrieval effectiveness over various baselines. In particular, PLAID-PRF improves over PLAID by up to 4.3% nDCG@10 and 7.3% MRR@10, while introducing substantially less computation overhead than prior PRF methods. The results demonstrate that our proposed centroid-aware PRF method offers an effective and lightweight mechanism to improve the quality of top-ranked retrieved results. Overall, this work enables effective and efficient feedback-aware late-interaction retrieval without expensive query-time document-token clustering.

CCS Concepts

• Information systems → Retrieval models and ranking.

Keywords

Dense Retrieval; Pseudo-Relevance Feedback

ACM Reference Format:

Xiao Wang, Sean MacAvaney, and Craig Macdonald. 2026. PLAID-PRF: Pseudo-Relevance Feedback with Centroid-like Tokens in PLAID. In *Proceedings of the 49th International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR ’26)*, July 20–24, 2026, Melbourne, VIC, Australia. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3805712.3809690>



This work is licensed under a Creative Commons Attribution 4.0 International License. *SIGIR ’26, Melbourne, VIC, Australia*
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2599-9/2026/07
<https://doi.org/10.1145/3805712.3809690>

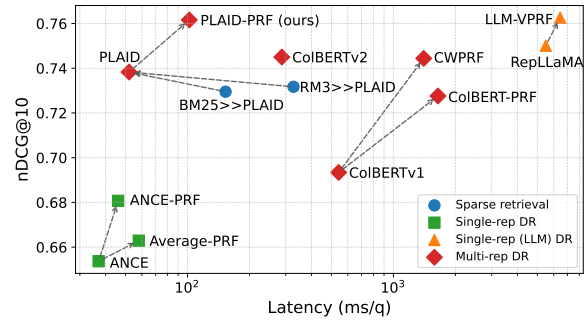


Figure 1: Effectiveness vs. efficiency trade-off on TREC Deep Learning 2019. The y-axis reports nDCG@10, and the x-axis reports query latency (ms/q) on a log scale.

1 Introduction

Dense retrieval models encode different granularities of signals into dense (embedding) vectors depending on their architecture. A primary way of categorising these models is based on what each vector represents: single-vector models encode one vector per retrieval unit (usually passages), while multi-vector models encode one vector per sub-retrieval unit (usually tokens). Compared to single-vector models, multi-vector models are capable of performing fine-grained semantic matching over individual tokens—a capability that is lost during the pooling stage of single-vector models. For instance, the multi-vector model ColBERT [18] and its derivatives (e.g., [5, 6, 21, 26, 40, 47]) perform *late interaction* to estimate query-document relevance: the tokens of a query and document are independently encoded using a heavy-weight language model, and relevance is estimated using light-weight token-level similarity aggregation (e.g., MaxSim). This process acts like a simple attention mechanism that allows the relevance score to depend only on the parts of each document that are most related to the query, which is a capability missing from single-vector dense models because all tokens are pooled into one.

The main drawback of multi-vector models is that they incur substantial storage and memory overheads due to having many vectors per document (usually one per token). ColBERTv2 [40] addressed these drawbacks by clustering and compressing vectors during indexing. Specifically, each token’s vector is compressed into the ID (*code*) of its nearest centroid together with a quantised residual, yielding 20-32 bytes per vector (depending on the settings), down from 256 bytes without compression. Later, PLAID [39] leveraged these compressed vectors to further reduce approximate retrieval costs by progressively filtering candidate documents through increasingly precise estimations of the model’s relevance scores.

An orthogonal approach to improve retrieval effectiveness is Pseudo-Relevance Feedback (PRF). PRF typically consists of three phases: (1) initial retrieval for a query; (2) reformulation of the query’s representation based on the top-ranked results; and (3) execution of this refined query representation [10]. PRF approaches have historically relied on lexical evidence from the corpus, such as word co-occurrences [15, 20]; More recently, PRF approaches have evolved to incorporate semantic signals, for instance by leveraging dense semantic vectors [22, 49] and large language models [24], to refine the query representation for a second retrieval pass. PRF methods have also been successfully designed for multi-vector retrieval models, most notably ColBERT-PRF [46] and CWPRF [43]. Although highly effective, these methods require computationally expensive clustering or additional model inference to select useful expansion vectors to add to the query, which substantially increases response times. In the standard three-stage PRF process, reducing the cost of the reformulation step is therefore crucial for the overall efficiency of the method, and its deployability in practical settings.

The key insight of this work is that we can leverage the existing indexing-time clusters that PLAID uses for index compression and retrieval to reduce the cost of PRF-based query reformulation. To this end, we introduce PLAID-PRF, a training-free PRF framework that directly uses PLAID’s centroid vectors. More specifically, (1) we mine informative centroids from the initial top-retrieved set; (2) we estimate their usefulness using classical term-weighting models (e.g., TF-IDF), computed from *centroid* occurrence statistics; and (3) we expand the original query with a small set of informative and diverse vectors from pseudo-relevant documents and execute the new query with PLAID. We find that our proposed PLAID-PRF framework, the first PRF work specifically for PLAID, is both inexpensive and effective, setting a new Pareto-optimal operational point as shown in Figure 1. In particular, PLAID-PRF improves over PLAID up to +4.3% in terms of nDCG@10 on DL’19, and on DL-HARD, it yields a +7.3% gain in MRR@10, while retaining low-latency retrieval (102 ms/query).

2 Related Work

Pseudo-relevance feedback (PRF) is a long-standing family of methods which leverage signals from the top-scoring documents from retrieval to inform the rest of the process [33]. PRF methods typically¹ perform automatic query reformulation in three phases: (1) retrieval over the corpus in an initial retrieval stage using the original query; (2) reformulation of the query representation based on the results from the first stage (e.g., by re-weighting and expanding terms in the query); and (3) executing the reformulated query for the final results. PRF techniques can be organised into two families: *Discriminative PRF* (Section 2.1) and *Generative PRF* (Section 2.2) techniques. Discriminative PRF methods select and apply weights to the information already present in the feedback documents (e.g., the expansion terms chosen in the RM3 [15] technique), and interpolate them with the original query. In contrast, generative PRF methods produce additional text or vectors, often generated by Large Language Models (LLMs), to augment the original query with terms that are not necessarily in the feedback documents.

¹ A separate class of PRF methods, Adaptive Retrieval, perform only a single stage of retrieval and do not reformulate the query. Instead, they use pseudo-relevance signals to select which documents from the corpus to score [28, 36, 37]. These methods are beyond the scope of this work.

2.1 Discriminative PRF

Classical sparse PRF. Classical lexical-based PRF methods are largely grounded in probabilistic and heuristic techniques. Notable algorithms include the RM1 [20] and RM3 [15] models. RM1 forms the foundation by estimating a feedback language model directly from the top-k documents to identify likely expansion terms. Building upon RM1, RM3 linearly interpolates the RM1 model with the original query, which helps mitigate query drift. Other influential classical techniques include the Divergence From Randomness (DFR) framework, which includes models like Bo1 [2] that weight terms by the divergence of their distribution in the top-k documents from the corpus distribution. Methods based on TF-IDF weighting also form a classical basis for selecting salient terms. These approaches remain strong sparse baselines and are often used as first-stage candidate generators for neural rerankers (e.g., BM25+RM3 followed by BERT/ColBERT reranking) [12, 14, 46]. However, classical methods operate over bag-of-words evidence and cannot exploit rich contextual information in feedback documents (e.g., disambiguating senses of polysemous words). This limitation motivated neural PRF for lexical pipelines, where contextual encoders help select more informative expansions.

Neural PRF for sparse retrieval. Neural PRF methods leverage distributed representations to capture semantic relationships. Early embedding-based query expansion methods also explored the use of distributed representations in pseudo-relevance feedback settings [50]. Later, BERT-QE [51] and CEQE [34] employed BERT-derived contextual embeddings to select more semantically-aware expansion terms. While these methods improve effectiveness, they still operate within a lexical framework, leaving a semantic gap between the selected terms and the underlying relevance signal. Indeed, because they still expand queries in the lexical space, their improvements are ultimately bounded by term selection. This motivates dense PRF methods that perform feedback directly in the embedding space.

Dense PRF. To overcome limitations of the lexical space, dense PRF expands the query using dense semantic vectors. In single vector retrieval, ANCE-PRF [49] re-encodes the query together with the PRF documents to produce a refined query vector. Vector-PRF [22] refines the dense query vector by aggregating the vectors of the top-ranked pseudo-relevant documents (e.g., via averaging or Rocchio-style interpolation). In contrast, ColBERT-PRF [46] is designed for multi-vector settings: a first pass retrieves with late interaction; feedback token vectors are clustered in embedding space using KMeans; representative centroids are selected and weighted; a small set of these vectors are added to the query before re-execution. Follow-ups such as CWPRF [43] learn contrastive weights to select useful expansion vectors. By providing additional query vectors that are contextualised within the feedback documents, these techniques provide improved semantic matching – allowing to retrieve further relevant documents even if they are lexically disparate from the feedback documents. However, while ColBERT-PRF and CWPRF can effectively expand the query, they are costly: ColBERT-PRF relies on an expensive clustering of the (large) token vectors of pseudo-relevant documents, while CWPRF (like ANCE-PRF) invokes a new transformer model to compute the feedback vectors.

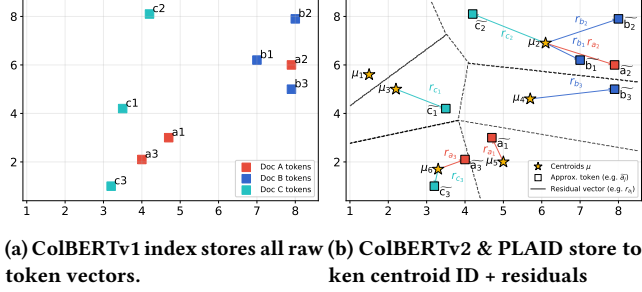


Figure 2: Indexing in ColBERTv1 and ColBERTv2/PLAID.

Instead, we seek an inexpensive PRF stage that can operate effectively within the quantised vector space used by ColBERTv2/PLAID.

2.2 Generative PRF

Generative PRF methods focus on using a generative model to produce expansion content conditioned on the query and/or feedback, rather than estimating term importance via explicit reweighting. These approaches often leverage generative models, including Large Language Models (LLMs). Broadly, they include query-side generation (e.g., GenPRF/GenQR [44] and LLM-based rewrites) or a set of explanatory paragraphs (as in GRF [30] and LLM-VPRF [24]) that are embedded or filtered to guide the second retrieval pass. HyDE [13] is a closely-related generation approach that synthesises a hypothetical passage to guide retrieval based on the user’s query; however it is not PRF since it does not use pseudo-relevant feedback documents. While these methods demonstrate a remarkable capacity for semantic capture and can handle complex queries, the generated content can be verbose or non-specific, so effectiveness on standard retrieval benchmarks is mixed, especially when the first-stage retriever is already strong and in-domain. Generative PRF is therefore promising—especially under domain shift—but it tends to increase inference cost and reduce control over the generated expansions, while expanding queries with more diverse terms and paraphrases. In this work, we focus on lightweight PRF for quantised late-interaction retrieval, where expansions are identified directly from indexed representations without extra generative inference; generative approaches are orthogonal and are left for future work.

3 Preliminaries

In this section, we describe the background knowledge of late interaction and explain the ideas that motivate our method using notation summarised in Table 1. We start with the late-interaction mechanism introduced in ColBERT [18]. We then describe how ColBERTv2 [40] compresses the document tokens while keeping the same retrieval procedure. After that, we outline the PLAID [39] retrieval pipeline.

Late interaction in ColBERT. ColBERT [18] is a dense retrieval model that scores based on the token-level vectors. More formally, given a query Q and a document D ,² ColBERT encodes them into token embeddings: $Q = [\phi_{q_1}, \dots, \phi_{q_{n_q}}] \in \mathbb{R}^{n_q \times d}$ and $D = [\phi_{d_1}, \dots, \phi_{d_{n_d}}] \in \mathbb{R}^{n_d \times d}$, respectively.

At retrieval time, ColBERT does not pool these embeddings into a single vector. Instead, it scores using a late-interaction operator

² For simplicity, we refer to all retrieval units (documents, passages, etc.) as documents.

Table 1: Summary of notation.

Notation for ColBERT / ColBERTv2 / PLAID	
ϕ_{q_i}, ϕ_{d_j}	Vector of the i -th query token q_i and j -th document token d_j (L2-normalised).
$\tilde{\phi}_{d_j}$	Approximate (reconstructed) vectors of the j -th document token d_j .
$Q = [\phi_{q_1}, \dots, \phi_{q_{n_q}}]$	Query representation as a sequence of token vectors; length n_q .
$D = [\phi_{d_1}, \dots, \phi_{d_{n_d}}]$	Document token sequence after ColBERTv1.
$\text{Score}(Q, D)$	Late-interaction score calculated using Equation (1).
$\Psi = \{\mu_c\}_{c=1}^C$	ColBERTv2 codebook (centroid vectors), C is the number of clusters.
$c_j \in \{1, \dots, C\}$	Centroid (code) ID assigned to document token d_j .
$\tilde{r}_j$	The quantised residual vector of document token d_j .
Notation for PLAID-PRF (our work)	
$\mathcal{P}_{\text{prf}}$	PRF set: top documents retrieved by a short first pass (stage-1 with the original query).
$\Phi = \{\tilde{\phi}_d\}_{d \in \mathcal{P}_{\text{prf}}}$	PRF token vectors gathered from $\mathcal{P}_{\text{prf}}$.
$\mathbf{c} = \{c(\tilde{\phi}_d)\}$	The centroid codes of tokens in Φ (one code per token).
f_p	Number of PRF documents.
f_e	Number of expansion vectors selected.
$\text{tf}(c)$	Frequency of centroid code c among PRF tokens Φ .
$\text{df}(c)$	Global document frequency for code c (precomputed).
$\mathcal{G}$	Weights of feedback tokens.
$E \subset \Phi$	Selected expansion vectors ($ E = f_e$).
β	Scaling of expansion vectors when appended to the query ($0 < \beta \leq 1$).
Q_{new}	Expanded query: $Q_{\text{new}} = [Q \oplus \beta \mathbf{e}_1 \oplus \dots \oplus \beta \mathbf{e}_m]$.
λ	Diversity weight in MMR selection.

that matches each vector in the query to the closest vector in the document:

$$\text{Score}(Q, D) = \sum_{i=1}^{n_q} \max_{1 \leq j \leq n_d} \text{Sim}(\phi_{q_i}, \phi_{d_j}), \quad (1)$$

This architecture allows fine-grained token-level matching and scores the document based on the tokens that are most semantically similar to the query tokens. In addition, since the document token vectors do not depend on the query, they can be computed offline.

ColBERTv2 residual representation. ColBERTv2 [40] retains the late-interaction scoring of ColBERT but compresses document vectors to make indexing and search efficient. ColBERTv2 learns a global *codebook* of centroid vectors $\Psi = [\mu_c]_{c=1}^C$, with each $\mu_c \in \mathbb{R}^d$. Here, c denotes the ID of each centroid, and C is the number of centroids in the codebook and d is the vector dimension. For a document token d_j , the indexer stores its nearest-centroid ID $c_j \in \{1, \dots, C\}$ together with a quantised residual, $\tilde{r}_j \in \mathbb{R}^d$, that encodes the difference of the vector from its centroid ($\phi_{d_j} - \mu_{c_j}$). At search time, ColBERTv2 reconstructs a close approximation of the original token vector by adding the centroid back to the residual representation:

$$\tilde{\phi}_{d_j} \approx \mu_{c_j} + \tilde{r}_j. \quad (2)$$

The codebook Ψ is stored once with the index; individual tokens carry only the integer c_j and their compressed residual.

The residual remains d -dimensional, but each dimension is quantised to a few bits (e.g., 1–2), then decompressed at query time for approximate reconstruction. The centroid-oriented nature of the resulting index enables efficient candidate generation, followed

by residual-aware refinement to preserve accuracy. This residual representation-based index structure also yields a substantial reduction in index size: on MS MARCO passages, ColBERTv2 achieves a 6–10 $\times$ compression ratio, reducing the index from 154 GiB to 16 GiB (1 bit per dimension) or 25 GiB (2 bits per dimension). As we argue in Section 4, this approximate representation of the document vectors has two key implications for our work. First, every document token is associated to a given centroid ID c_j – we observe that occurrences of *centroids* are countable, and hence we can measure their informativeness. Second, because centroids form the backbone of the index, reasoning at the centroid level is fast and stable: code-level signals are cheap to compute and align with how the system stores and accesses vectors. This suggests that use of (pre-computed) centroids within PRF may bring significant efficiency benefits over online computation of clusters used in ColBERT-PRF.

PLAID search pipeline. To further increase the retrieval efficiency of ColBERTv2, PLAID [39] searches over encoded ColBERTv2 vectors in a multi-stage cascading pipeline. First, it identifies a large candidate set by scoring the centroid vectors against the query (i.e., performing $\Psi \cdot Q^T$) and mapping the top-scoring centroids for each query vector back to the documents that contain them using an inverted index. It then produces increasingly fine-grained approximations of each document’s score, truncating the ranked list after each approximation. The final stage performs final MaxSim scoring over the reconstructed document vectors. The process is controlled by several parameters, which are studied in detail in [27].

The three architectures above, namely ColBERTv1 & v2 and PLAID, point to a clear path towards PLAID-PRF. Firstly, the summative nature of MaxSim allows additional query vectors obtained by PRF to improve effectiveness, as demonstrated by ColBERT-PRF & CWPRF. Secondly, ColBERTv2 provides a useful semantic unit – the centroid code – for identifying possible expansions; Thirdly, the PLAID pipeline reveals where extra query vectors generate the most value: by enabling the query to probe more centroids, which can uncover additional relevant documents, while also improving the precision of the top-ranked results.

4 PLAID-PRF

In this section, we introduce PLAID-PRF, which is illustrated in Figure 3. PLAID-PRF augments PLAID’s late-interaction retrieval with a small set of weighted expansion vectors derived from the initial top-retrieved documents. We describe PLAID-PRF in four stages, as highlighted in the figure: ① an initial retrieval; ② estimating the informativeness of centroids occurring in the retrieved documents; ③ identify a set of expansion vectors associated to the informative centroids to form the refined query representation; and ④ a final second-pass retrieval. Table 1 provides an overview of our used notation, while Sections 4.1–4.4 provide an overview of the four stages of PLAID-PRF.

4.1 Stage 1: Initial Retrieval

Given a tokenised query, the encoder produces a sequence of vectors $Q = [\phi_{q_1}, \dots, \phi_{q_{n_q}}]$, we perform an initial PLAID search to obtain a *feedback set* of the top- f_p documents, $\mathcal{P}_{\text{prf}} = \{p_1, \dots, p_{f_p}\}$. Further, by concatenating all $\mathcal{P}_{\text{prf}}$ documents, we can obtain a sequence

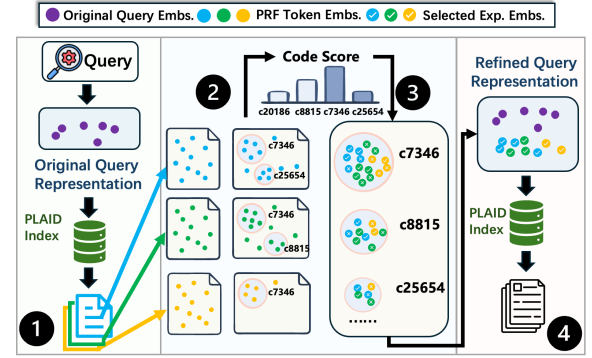


Figure 3: Overview of PLAID-PRF. An initial PLAID retrieval returns pseudo-relevant documents, where the feedback token vectors are stored in the PLAID index as centroid codes with quantised residuals. PLAID-PRF groups the tokens by their centroid code (e.g. c7346/c8815) and calculates the code-level score. PLAID-PRF ranks centroids by their scores and assigns higher usefulness to tokens originating from higher-scored centroids. Finally, a small set of highly-useful expansion vectors is selected and expanded to form a refined query representation for final retrieval.

of *feedback tokens*. In a PLAID/ColBERTv2 index, each token’s vector is associated with a centroid code c_j (from the indexing-time codebook) along with a quantised residual r_j used for reconstruction. These are used for retrieval, and available for PRF.

4.2 Stage 2: Usefulness Scoring

In PRF, we typically assume that the top-ranked feedback documents are relevant. However, they likely also contain redundant and noisy tokens, so naively appending all feedback token vectors can drift the query representation away from its original intent. Therefore, within the feedback token set, we estimate the usefulness of each token and incorporate the vectors of the most discriminative tokens into the query representation.

An informative token (and its corresponding centroid code) should be *discriminative* in two senses: it should occur frequently among the top feedback documents for this query, reflecting *topical relevance*, yet appear rarely across the collection overall, reflecting *specificity*. In ColBERT-PRF [46], the expansion mechanism implicitly balances topicality and specificity: topical relevance is captured by query-conditioned cluster centroids formed over pseudo-relevant token vectors, while specificity is measured by ranking clusters using the IDF of the nearest token to each centroid.

We observe that tokens sharing a centroid tend to be projected in a similar region of the vector space. This implies that, instead of performing expensive retrieval-time clustering (as in ColBERT-PRF), we can reason in terms of ColBERTv2 centroid occurrences, which are already stored in the index. A centroid that occurs frequently in the feedback documents can be considered useful for query reformulation. We therefore compute usefulness weights at the centroid level and propagate these weights to the tokens assigned to each centroid.

Indeed, instead of counting the specificity of tokens directly, the centroid codes enable us to approximate the discriminative

properties of each feedback token. In other words, we can compute classical IR statistics (TF, IDF, etc.) upon the occurrences of centroids.³ Unlike the classic PRF term weighting method over lexical terms, PLAID-PRF focuses on centroid IDs: every occurrence of the centroid code c in the PRF set can be quantified as $\text{tf}(c)$, and its global rarity can be measured using a centroid-level IDF. The latter is easily computed offline using the PLAID index.

Then, we map these weights back to the tokens based on the positionally aligned assignments between centroid codes and tokens. We provide further details of centroid weighting and token-level usefulness scores below.

Centroid weighting. Let Φ denote the sequence of all feedback token vectors in $\mathcal{P}_{\text{prf}}$, i.e. $\Phi = [\tilde{\phi}_{d_j}]_{j=1}^{|\Phi|}$ for each feedback token j , along with corresponding centroid codes $[c_j]_{j=1}^{|\Phi|}$. For each centroid c , we can count $\text{tf}(c) = \sum_{j=1}^{|\Phi|} \mathbb{1}(c_j = c)$, i.e., the number of feedback tokens assigned to centroid c for the current query. Similarly, we can pre-compute document frequency $\text{df}(c)$ at indexing time, i.e. how many documents in the index contain at least one token assigned to centroid c . This allows us to calculate a TF-IDF weight for the occurrence of each centroid within the feedback set, as: $w_{\text{tfidf}}(c) = \text{tf}(c) \cdot \frac{N+1}{\text{df}+1}$. Intuitively, TF-IDF gives high weights to centroids that are common in the feedback (indicating topical relevance) but rare in the corpus⁴ (indicating specificity) — and therefore capturing the discriminative nature of each cluster. As we show later, this repurposing of existing term weighting models applied to centroid occurrences works pleasantly well. Indeed, in our experiments, we investigate various classical feedback usefulness weighting models in Section 6.5.

Token-level usefulness. Given the per-centroid usefulness weights $w(c_j)$ computed above, each feedback token is assigned a final score, $\mathcal{G} \in \mathbb{R}^{|\Phi|}$ using the weight of its corresponding centroid code:

$$\mathcal{G} = [w(c_j)]_{j=1}^{|\Phi|}. \quad (3)$$

$\mathcal{G}$ is used to determine which of the reconstructed feedback vectors Φ to add back into the query representation, as described next.

4.3 Stage 3: Expansion Vector Selection

Given the (reconstructed) feedback vectors Φ and their token-level usefulness scores $\mathcal{G}$ (Section 4.2), we select a compact set of f_e expansion vectors $E \subset \Phi$ to append to the query representation. A naive Top- k policy picks the f_e tokens with the largest scores in $\mathcal{G}$. However, as centroids can represent more than one feedback token, this policy can result in very similar (i.e. redundant) vectors being selected. To encourage a more diverse set of expansion vectors, we adopt greedy maximal marginal relevance (MMR) [4, 7, 41], which greedily selects vectors into E that are highly weighted in $\mathcal{G}$ yet dissimilar to previously selected vectors and the original query vectors:

$$j^* = \arg \max_{j \notin E_t} \left[(1 - \lambda) \mathcal{G}_j - \lambda \max_{v \in E_t \cup Q} \text{Sim}(\tilde{\phi}_{d_j}, v) \right], \quad (4)$$

where we select an expansion vector indexed by j^* at each iteration step t , E_t is the current selected set, Q denotes the original query

³ Indeed, prior work has also shown that corpus-statistics-based reweighting of centroid vectors can be useful for retrieval using embedding-based representations [3, 11].

⁴ We did observe that the $\text{df}(\cdot)$ distribution exhibits approximately power-law-like behaviour over a limited range of frequencies ($10^3 \dots 10^4$).

vectors, and $\lambda \in [0, 1]$ controls the usefulness–diversity trade-off.⁵ We update $E_{t+1} = E_t \cup \tilde{\phi}_{d_{j^*}}$ and iterate until $|E_t| = f_e$. Notably, the expansion vectors E are formed of reconstructed vectors $\tilde{\phi}_{d_j}$, rather than centroid vectors μ_j — indeed, as we show later, centroid vectors are too coarse for effective query refinement.

4.4 Stage 4: Expand the query and rerun PLAID

We append the selected expansion vectors E to form the refined query representation:

$$Q_{\text{new}} = [Q \oplus \beta \cdot E] \in \mathbb{R}^{(n_q + f_e) \times d}, \quad (5)$$

where $\beta \in (0, 1]$ controls the feedback contribution. We then rerun the PLAID retrieval using the refined query representation.

4.5 Discussion

Dense PRF methods such as ColBERT-PRF and CWPRF operate directly on the full-precision token vectors of ColBERTv1. In contrast, ColBERTv2 avoids the intractable memory requirements of storing such vectors by relying on quantisation, which precludes direct access to full-precision token representations. As a result, PLAID-PRF is designed for quantised late interaction and operates over reconstructed vectors. Specifically, it estimates topical relevance using PLAID’s *centroid codes*, treating centroid IDs as semantic “terms”, and subsequently maps the resulting weights back to the corresponding reconstructed token vectors. This design allows PLAID-PRF to reuse PLAID’s compressed, centroid-based index structure and multi-stage probing without retraining the retriever, while still supporting term-based weighting schemes and MMR-style selection over reconstructed expansion vectors. Central to this design are three modelling choices: leveraging PLAID’s indexing-time clusters rather than computing retrieval-time clusters as in ColBERT-PRF; counting centroid frequencies instead of WordPiece occurrences; and performing expansion over reconstructed vectors rather than directly using centroids. To our knowledge, PLAID-PRF is the first PRF framework that is both *codebook-aware* and *quantisation-compatible*, aligning feedback statistics with the structure of a ColBERTv2/PLAID index. Our experiments aim to validate our three modelling choices, as described next.

5 Experimental Setup

Our first experiments focus upon the key modelling decisions in PLAID-PRF:

RQ1. Is feedback based on indexing-time clustering more effective than feedback based on PRF-stage clustering?

RQ2. Does codebook-based centroid counting yield more effective expansions than using WordPiece token statistics?

RQ3. Is the use of the reconstructed token vectors necessary for effective PRF, or can raw centroid vectors be used for expansion? Then, we compare the retrieval performance of PLAID-PRF to various existing baselines and using out-of-domain benchmarks:

RQ4. How does PLAID-PRF compare against state-of-the-art retrievers and PRF methods in effectiveness and latency?

Furthermore, we conduct a sensitivity analysis for PLAID-PRF:

⁵ We min-max normalise $\mathcal{G}$, such that they lie in a compatible range with cosine similarity $\text{Sim}(\cdot)$.

RQ5. How sensitive is PLAID-PRF to (i) the expansion settings (number of feedback documents etc.); (ii) choice of centroid weighting model, and (iii) diversity-based expansion selection setting?

RQ6. Which expansion vectors help or hurt PLAID-PRF?

RQ7. Does PLAID-PRF generalise to out-of-domain retrieval, as measured on selected BEIR datasets?

5.1 Setup

Datasets and Metrics. Experiments are conducted in-domain on MS MARCO passage corpus [35], using the TREC Deep Learning 2019 [9] and 2020 [8] query sets. The corpus contains 8.8M passages gathered from Bing search results. For TREC DL 2019 and 2020, we use the 43 and 54 topics with multi-grade relevance assessments, respectively. On average, these test collections provide around 95.4 and 66.8 relevance judgements per query for 2019 and 2020 respectively, giving sufficiently deep labels to support the evaluation of pseudo-relevance feedback methods [22, 46, 49].⁶ Moreover, we also evaluate using DL-HARD [31] query sets, which consist of 50 most challenging DL topics according to the SERP-based criteria. We focus on in-domain TREC DL benchmarks to isolate the effect of our codebook-based expansion under a controlled evaluation setting. Moreover, we report results on four representative test collections from BEIR benchmark suite [42], namely DBPedia, NF-Corpus, TREC-COVID, and Touché-2020. Following prior work [45], we restrict evaluation to BEIR test collections with a sufficiently large number of relevance judgements per query, as measurements of pseudo-relevance feedback are known to be less reliable on test collections with sparse judgements [1].

Following the standard practice of recent dense retrieval works, we report mean reciprocal rank (MRR) and normalised discounted cumulative gain (nDCG) calculated at rank 10 as our primary effectiveness metrics, together with Recall and mean average precision (MAP) calculated at rank 1000. For binary metrics, namely MRR, MAP, and Recall, we consider labels of two or greater to be relevant [9]. For efficiency, we report the mean response time (MRT), i.e., the average per-query latency (ms/query) measured with a query batch size of 1. For statistical significance testing, we use the paired t-test with Holm–Bonferroni multiple testing correction ($p < 0.05$). For the BEIR evaluation, we report nDCG@10 and Recall@1000 to compare both early ranking quality and high-depth retrieval coverage [17, 42].

Parameters. For PLAID-PRF, we use the official ColBERTv2 checkpoint,⁷ trained on the MS MARCO passage ranking corpus [35], and build a PLAID index with residual compression. Each token’s vector is stored as a centroid identifier plus a quantised residual, using 2-bit residual quantisation ($n_{bits}=2$). For retrieval, we enable the PLAID engine and adopt the recommended operational point from [27, 39], corresponding to ($n_{cells} = 4$, $t_{cs} = 0.40$, $n_{docs} = 4096$). Based on the parameter study in Section 6.5, we tune PLAID-PRF on TREC DL’19 via a grid search over f_p , f_e , β , λ and fix the parameters for all other test sets. We select the configuration based on nDCG@10 and set $f_p = 3$ feedback passages, $f_e = 14$ expansion vectors, $\beta = 0.7$ and

$\lambda = 0.3$. For the BEIR experiments, we keep the structural PLAID-PRF parameters fixed and tune only the feedback scaling factor β . This is a practice analogous to the interpolation weight commonly used in classical PRF methods such as Rocchio- and RM3-style feedback [15, 38]. For estimating the usefulness of centroids, in addition to TF-IDF, we also investigate other heuristic weighting methods, including RM1 [20], RM3 [15] and Bo1 [2], as well as a strawman random weighting of centroids. All experiments of PLAID-PRF are conducted using a single NVIDIA GeForce RTX 3090 GPU. The source code for reproducing PLAID-PRF is available in the PyTerrier_ColBERT2 repository.⁸ We also provide a virtual appendix containing the result files to produce the result tables in this paper.⁹

5.2 Baselines

We compare PLAID-PRF with the following 4 families of retrieval systems, applied with and without PRF.

Sparse Retrieval. We compare with the classical sparse retrieval systems, namely BM25, and with RM3 [15] and PRF technique implemented. We implement these systems using PyTerrier [29].¹⁰ Moreover, we also compare both BM25 and BM25 + RM3 systems with PLAID as a reranker, which are implemented following [27]. **Single-vector dense retrieval.** We include ANCE [48] as a standard dual-encoder dense retriever trained on MS MARCO, together with two PRF variants: ANCE-PRF [49], which applies classical term-based PRF on top of ANCE scores, and Average-PRF [22], which performs an average operation for the feedback vectors in the dense representation space.¹¹

Single-vector LLM-based dense retrieval. We compare with the more-powerful LLM-based dense retrieval approaches and their PRF extensions, namely PromptReps (Qwen 8B) [52] and Prompt-PRF (8B) [23] (which augments PromptReps with LLM-generated features). Moreover, we implement RepLLaMA (7B) [25] and its LLM-VPRF [24] variant with FAISS [16] HNSW [32] search, which applies vector-space PRF on top of RepLLaMA.¹² Together, these baselines represent the state of the art in single-rep LLM-centric dense retrieval and PRF.

Multi-vector dense retrieval. Finally, we compare with multi-vector models. ColBERTv1 [18] is the original late-interaction retriever operating on token-level representations; ColBERT-PRF [46] and CWPRF [43] extend it with PRF in the token space.¹³ We also compare with ColBERTv2 [40] and PLAID [39].¹⁴

6 Results and Analysis

We firstly address RQs 1-3 – concerned with design choices in PLAID-PRF – in Sections 6.1 - 6.3. In addressing these RQs, we make use of Table 2 (effectiveness of variants) and Figure 4 (efficiency of variants). Section 6.4 compares PLAID-PRF to state-of-the-art (cf. RQ4), while Section 6.5 reports the sensitivity analysis (cf. RQ5). Furthermore, Section 6.6 presents the qualitative analysis of PLAID-PRF (cf. RQ6) and Section 6.7 examines the out-of-domain generalisation of PLAID-PRF (cf. RQ7).

⁸ https://github.com/cmacdonald/pyterrier_colbert2/tree/main

⁹ <https://github.com/Xiao0728/PLAID-PRF-VirtualAppendix/tree/master>

¹⁰ <https://github.com/terrier-org/pyterrier>

¹¹ https://github.com/terrierteam/pyterrier_dr/

¹² As PLAID is a compressed ANN index representation, for fair comparison we similarly apply RepLLaMA using an ANN, specifically using HNSW with settings proposed in [19].

¹³ https://github.com/terrierteam/pyterrier_colbert

¹⁴ <https://github.com/seanmacavaney/plaidrepro>

⁶ MS MARCO dev set offers very shallow annotations (on average, 1.1 relevant passages per query); PRF can be unreliable on such sparsely judged test collections [1, 45], therefore we omit the Dev queries. ⁷ <https://huggingface.co/colbert-ir/colbertv2.0>

Table 2: Effectiveness of PLAID-PRF Variants. † indicates a significant improvement over PLAID, while ‡ indicates the variant is significantly outperformed by PLAID-PRF (paired test, $p < 0.05$). The highest value in each column is boldfaced.

Methods	TREC 2019		TREC 2020		DL-HARD	
	nDCG@10	MRR@10	nDCG@10	MRR@10	nDCG@10	MRR@10
PLAID	0.7383	0.8934	0.7410	0.8411	0.4146	0.5697
LocalCluster-PRF	0.7523	0.8879	0.7552	0.8298	0.4160	0.5476
TokenCount-PRF	0.7610	0.9310	0.7573	0.8486	0.4258	0.5526
CentroidExp-PRF	0.7502	0.8820	0.7581	0.8294	0.4002‡	0.5046‡
PLAID-PRF	0.7700†	0.9287	0.7435	0.8488	0.4351†	0.6110

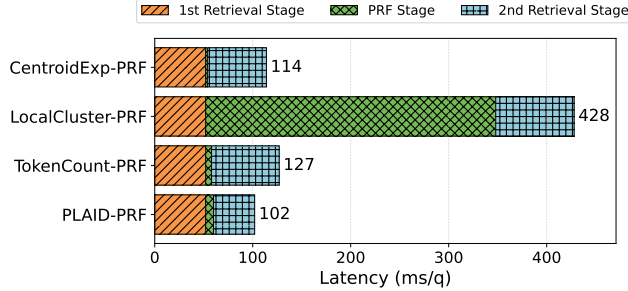


Figure 4: Latency Breakdown of PLAID PRF variants.

6.1 RQ1: Indexing-time clustering vs. PRF-stage clustering

Experimental Setup. We compare PRF based on *indexing-time clustering* (PLAID-PRF) against a PRF variant based on *PRF-stage clustering* (denoted LocalCluster-PRF). Indeed, LocalCluster-PRF is essentially ColBERT-PRF using a KMeans but using the reconstructed token vectors rather than exact token vectors (which are not stored in a ColBERTv2 index). The two approaches differ only in the clustering source – the global PLAID codebook with $k = 2^{18}$ clusters versus per-query KMeans with $k = 24$ – while keeping the 1st and 2nd pass retrieval configurations unchanged.¹⁵

Results. Table 2 shows that PLAID-PRF yields consistently higher early-precision effectiveness than LocalCluster-PRF. In particular, PLAID-PRF improves over LocalCluster-PRF on all reported metrics across the three query sets, for example, on DL’19, PLAID-PRF substantially improves nDCG@10 (0.7523 $\rightarrow$ 0.7700). Meanwhile, efficiency differences are even larger: Figure 4 reports 423 ms/query for PRF-stage clustering, versus 102 ms/query for PLAID-PRF (4.15 $\times$ faster), with most of the excess latency attributable to the runtime KMeans over feedback token vectors. We postulate that centroid-only expansions are less well aligned with PLAID’s codebook, which reduces selectivity and increases the amount of second-pass refinement and late-interaction scoring. This shows the advantage of using PLAID’s codebook, computed at indexing time, rather than an expensive KMeans clustering for each query at inference time. Overall, in response to **RQ1**, indexing-time clustering yields a markedly better effectiveness–efficiency trade-off.

6.2 RQ2: Counting centroids or tokens

Experimental Setup. Since the PLAID index stores token representations in compressed form (centroid codes with quantised

residuals) and does not retain the original WordPiece IDs, we pre-compute a mapping from each passage’s token positions to their WordPiece token IDs using the same BERT tokeniser as in PLAID indexing. This yields a strict one-to-one alignment between (i) the token text produced by the tokeniser, (ii) its WordPiece ID, and (iii) its position in the PLAID token sequence, enabling us to recover WordPiece IDs for the pseudo-relevant tokens returned in the first pass. We then implement a PRF variant, denoted TokenCount-PRF, that computes TF-IDF weights at the WordPiece level, rather than the centroid-code level as per PLAID-PRF.

Results. Table 2 shows that replacing WordPiece-level counting (TokenCount-PRF) with codebook-aware centroid counting (PLAID-PRF) yields consistently stronger retrieval effectiveness. This is because centroid counting treats PLAID’s centroid IDs as semantic “terms”, mapping multiple tokenisation variants and morphological forms to shared semantic units, which reduces redundant expansions and better aligns the feedback evidence with PLAID’s codebook representation. This tends to promote the most relevant documents earlier (higher MRR), whereas TokenCount-PRF can be sensitive to tokenisation and affixes. In terms of retrieval latency, as shown in Figure 4, PLAID-PRF is also faster than TokenCount-PRF (102 vs. 127 ms/q), suggesting that code-level counting not only improves ranking quality but also avoids the overhead associated with WordPiece-level token alignment in the retrieval pipeline. Overall, in response to **RQ2**, we find that codebook-based centroid counting yields more effective and more efficient expansions than WordPiece-level statistics.

6.3 RQ3: Reconstructed vectors vs. raw centroid vectors as expansion units

Experimental Setup. We examine whether the choice of expansion units matters when performing PRF in a PLAID setting. Specifically, we compare PLAID-PRF, which expands the query with reconstructed token vectors (centroid vectors plus quantised residual), against a centroid-only variant (denoted CentroidExp-PRF) that uses raw codebook centroids as expansion vectors.

Results. Table 2 shows that CentroidExp-PRF is competitive in nDCG@10, achieving the best DL’20 nDCG@10 (0.7581)—but yields lower early precision than PLAID-PRF on both reported metrics for DL’19 and DL-HARD query sets. This suggests that using raw centroids without residuals yields a coarser expansion signal: it can improve overall top-10 ranking (nDCG@10) yet is less effective at promoting the first relevant item to the very top ranks (MRR@10). In contrast, reconstructed token vectors retain centroid+residual information and better match MaxSim late interaction, yielding stronger MRR. This shows the necessity of using reconstructed vectors for effective expansion, answering **RQ3**. Finally, we note that CentroidExp-PRF shows a comparable end-to-end latency to PLAID-PRF (114 vs. 102 ms/q), but is consistently slower due to a heavier second-pass retrieval stage.

6.4 RQ4: PLAID-PRF vs. SOTA

In this section, we evaluate PLAID-PRF wrt. state-of-the-art retrievers in terms of both effectiveness and efficiency. Table 3 reports the main results across query sets, while Figure 1 visualises the

¹⁵ Following ColBERT-PRF [46], we set $k = 24$ for PRF-stage KMeans.

Table 3: Retrieval effectiveness and efficiency of PLAID-PRF and various baselines. A superscript $\dagger$ on a model’s score indicates that PLAID-PRF significantly outperforms that baseline (paired test, $p < 0.05$). The highest value in each column is boldfaced.

Methods	TREC 2019 (43 queries)				TREC 2020 (54 queries)				DL-HARD (50 queries)				Latency (ms)
	MAP	nDCG@10	MRR@10	R@1k	MAP	nDCG@10	MRR@10	R@1k	MAP	nDCG@10	MRR@10	R@1k	
Multiple Vector Dense Retrieval													
ColBERTv1 [18]	0.4318†	0.6934†	0.8527	0.7892†	0.4654†	0.6871†	0.8500	0.8245†	0.2358†	0.3824†	0.5291	0.7586†	542
ColBERT-PRF [46]	0.5295	0.7283	0.8508	0.8746	0.4904	0.6937†	0.7684†	0.8914	0.2306†	0.3683†	0.4546†	0.8021	1645
CWPRF [43]	0.5335	0.7444	0.8671	0.8596	0.5145	0.7246	0.8421	0.8783	-	-	-	-	1403
ColBERTv2 [40]	0.5118	0.7448†	0.8973	0.8940	0.5270	0.7401	0.8408	0.9129	0.2727†	0.4144†	0.5720	0.8655	287
PLAID [39]	0.5097	0.7383†	0.8934	0.8706	0.5262	0.7410	0.8411	0.9044	0.2727†	0.4146†	0.5697	0.8389	52
PLAID-PRF (Ours)	0.5282	0.7700	0.9287	0.8967	0.5236	0.7435	0.8482	0.8906	0.2905	0.4351	0.6110	0.8485	102
Sparse Retrieval Systems													
BM25	0.2864†	0.4795†	0.6396†	0.7553†	0.2930†	0.4936†	0.6147†	0.8072†	0.1471†	0.2743†	0.4151	0.6690†	80
BM25 + PLAID [27]	0.4815	0.7295	0.8760	0.7553†	0.5091	0.7435	0.8584	0.8072†	0.2520†	0.4000†	0.5372	0.6866†	153
BM25 + RM3	0.3192†	0.5247†	0.6462†	0.7859†	0.3161†	0.5107†	0.6141†	0.8333†	0.1646†	0.2843†	0.4095	0.7103†	253
BM25 + RM3 + PLAID	0.4942	0.7423	0.8798	0.7859†	0.5091	0.7401	0.8693	0.8333†	0.2560†	0.4080	0.5406	0.7103 †	327
Single Vector Dense Retrieval													
ANCE [48]	0.3716†	0.6210†	0.7876†	0.7234†	0.3871†	0.6077†	0.7460†	0.7423†	0.1839†	0.2980†	0.4364†	0.6892†	37
ANCE-PRF [49]	0.4296†	0.6409†	0.7829†	0.7237†	0.4090†	0.6160†	0.7594	0.7400†	0.1955†	0.2981†	0.4474†	0.6554†	58
Average-PRF [22]	0.4163†	0.6303†	0.7442†	0.7155†	0.4080†	0.6197†	0.7459†	0.7282†	0.1927†	0.2995†	0.4029†	0.6375†	46
Single Vector (LLM) Dense Retrieval													
PromptReps (8B) [52]	0.3165†	0.5422†	0.6940†	0.7377†	0.2734†	0.4960†	0.6549†	0.7282†	-	-	-	-	-
PromptPRF (8B) [23]	-	0.5941	-	0.7176	-	0.5265	-	0.7040	-	-	-	-	-
RepLLaMA (7B) [25]	0.4325†	0.6117†	0.7543†	0.7044†	0.4977	0.7241	0.8843	0.8189†	0.2628	0.4132	0.5343	0.7496†	5496
LLM-VRPF [24]	0.4409†	0.6300†	0.7775†	0.8905†	0.5222	0.7335	0.8525	0.8786†	0.2753	0.4378	0.5576	0.7791	6454

retrieval effectiveness-latency trade-off on DL’19. We first discuss effectiveness and then analyse latency.

Effectiveness of PLAID-PRF. From Table 3, we observe that PLAID-PRF delivers the strongest top-ranked effectiveness while retaining the efficiency advantages of PLAID-style approximate late interaction compared to various baseline families.

Firstly, we examine the effectiveness of the proposed PRF mechanism by comparing PLAID-PRF with PLAID. From Table 3, we find that PLAID-PRF yields significant improvements over PLAID. More specifically, it improves nDCG@10 by +4.3% (0.7383→0.7700) and MRR@10 by +4.0% (0.8934→0.9287) on DL’19 and even higher improvements on DL-HARD, with nDCG@10 increasing by +4.9% (0.4146→0.4351) and MRR@10 by +7.3% (0.5697→0.6110). These gains in early-precision metrics indicate that PRF expansions do not merely increase recall, but more importantly promote relevant documents to higher ranks. Similarly, PLAID-PRF significantly improves over ColBERTv1 and ColBERTv2 models. Moreover, PLAID-PRF also exhibits large gains over prior token-level PRF methods: on DL’20 it improves ColBERT-PRF by +6.3% nDCG@10 (0.6993→0.7435) and +5.5% MAP (0.4962→0.5236), and surpasses CWPRF by +2.6% nDCG@10 (0.7246→0.7435).

Furthermore, we also compare against strong sparse pipelines and single-vector dense models (both conventional and LLM-based). In particular, within sparse retrieval, PLAID-PRF improves over strong cascades, which employ PLAID as reranker in BM25+PLAID and BM25+RM3+PLAID baselines. Secondly, compared with single-vector dense retrievers (ANCE and its PRF variants), PLAID-PRF is significantly stronger on both DL’19 and DL’20 (e.g., DL’20 nDCG@10: 0.7435 vs. 0.6077 for ANCE, +22.3%), consistent with the known advantage of late interaction for fine-grained evidence alignment. Finally, recent LLM-based retrievers form a strong (often SOTA) baseline family. PLAID-PRF remains highly competitive in

this regime, typically outperforming prompt-based dense retrievers and RepLLaMA. This indicates that an efficient token-level PRF mechanism atop PLAID can rival LLM-based retrieval pipelines without requiring substantially heavier inference and computational resources.

Efficiency of PLAID-PRF. From Table 3 and Figure 1, PLAID-PRF achieves a strong effectiveness–efficiency balance and lies on the Pareto frontier on DL’19. Figure 4 further decomposes the end-to-end latency of PLAID-PRF into three stages: the 1st-pass retrieval, the PRF stage, and the 2nd-pass PLAID rerun. Overall, PLAID-PRF requires 102 ms/query, where the PRF stage contributes only a small fraction of the total runtime. This is because PLAID-PRF computes the expansion usefulness scores through lightweight centroid-level statistics and directly forms expansion vectors from the retrieved token representations. Importantly, the refined query representation is constructed in the vector space and passed to the second-stage PLAID rerun without re-encoding the query, so the overall cost remains dominated by the two PLAID retrieval passes rather than the feedback computation.

6.5 RQ5: Sensitivity Analysis

Impact of the PRF parameters. Figure 5 studies four PRF parameters with clear operational meanings: f_p controls how many pseudo-relevant passages contribute feedback evidence, f_e controls how many expansion vectors are appended to the query, β controls the interpolation strength of the expansion and λ controls the relevance–diversity trade-off in MMR selection. Across all settings, many configurations exceed the effectiveness of PLAID baseline (dashed line), showing that PLAID-PRF’s gains are not confined to a single fragile point.

More specifically, in terms of f_p and f_e (Figure 5a), increasing f_p from 1 to 3 improves nDCG@10, as more top-ranked passages

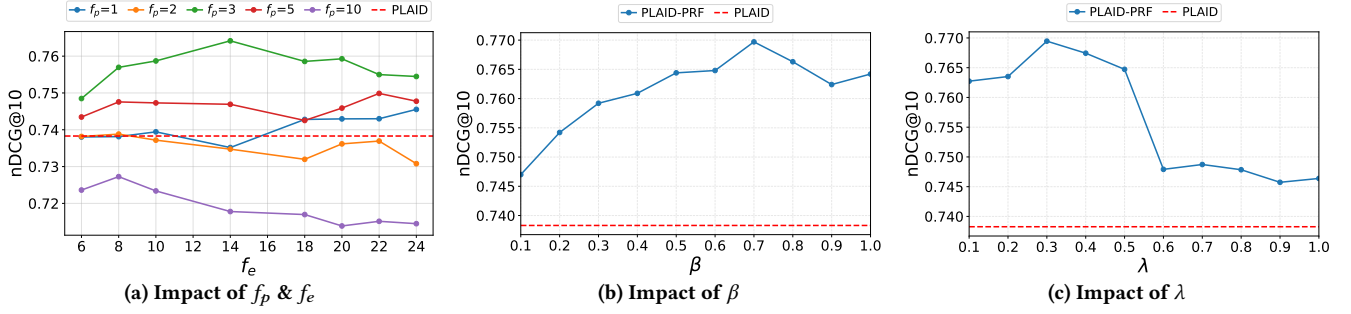


Figure 5: Impact of f_p , f_e , β and λ on the Retrieval Effectiveness (nDCG@10 on TREC 2019 queries) of PLAID-PRF.

Table 4: Impact of the weighting method & expansion selection method for PLAID-PRF. Notations per Table 2

Methods	TREC 2019		TREC 2020		DL-HARD	
	nDCG@10	MRR@10	nDCG@10	MRR@10	nDCG@10	MRR@10
PLAID	0.7383	0.8934	0.7410	0.8411	0.4146	0.5697
Impact of weighting method						
PLAID-PRF (Rand. W.)	0.7362 $\ddagger$	0.8632 $\ddagger$	0.7471	0.8127	0.4179	0.5319
PLAID-PRF (RM1)	0.7616	0.9171	0.7561	0.8482	0.4367$\dagger$	0.5831
PLAID-PRF (RM3)	0.7630	0.9132	0.7549	0.8575	0.4365 $\dagger$	0.5847
PLAID-PRF (Bo1)	0.7659	0.9054	0.7434	0.8482	0.4316	0.5874
PLAID-PRF (TF-IDF)	0.7700$\dagger$	0.9287	0.7435	0.8488	0.4351 $\dagger$	0.6110
Impact of expansion selection method						
PLAID-PRF (Rand. Sel.)	0.7413	0.8740	0.7555	0.8480	0.4140	0.5476
PLAID-PRF (Top-k)	0.7594	0.9132	0.7439	0.8520	0.4274	0.5938
PLAID-PRF (MMR)	0.7700$\dagger$	0.9287	0.7435	0.8488	0.4351$\dagger$	0.6174

provide a broader feedback signal, however, at larger f_p values (e.g. 10), nDCG@10 degrades due to noisier or less relevant feedback. This trend is consistent with findings in classical PRF [2, 15] and dense-PRF [46, 49], where small feedback depths are often preferred to limit query drift. For f_e , performance rises with more expansions but saturates and can regress when f_e becomes large, reflecting diminishing returns and redundancy among expansion vectors. The curve peaks around $f_p = 3$, $f_e = 14$, which we adopt. In terms of β and λ (Figure 5b & Figure 5c, respectively), we observe that nDCG@10 increases as β rises from 0.1 to 0.7, then slightly decreases as β increases further (0.8–1.0). The best DL’19 point occurs near $\beta = 0.7$, but $\beta \in [0.4, 0.7]$ remains consistently strong, so we treat this as a robust range. λ peaks at a small value ($\lambda = 0.3$: mild diversity regularisation helps avoid near-duplicate expansions, while larger λ over-emphasises diversity and hurts relevance. Overall, these trends motivate our final settings: $f_p = 3$, $f_e = 14$, $\beta = 0.7$ (robustly $\beta \in [0.4, 0.7]$), and $\lambda = 0.3$.

Impact of the weighting method. Table 4 reports ablations on the weighting scheme and the expansion selection strategy in PLAID-PRF. We find that using random weights eliminates the benefits of PRF: effectiveness becomes unstable and early precision degrades, indicating that gains come from meaningful token importance estimation rather than simply appending extra vectors. In addition, replacing TF-IDF with alternative weighting methods shows that several choices remain competitive, but TF-IDF provides the most robust gains overall: it achieves the strongest effectiveness on DL’19 (nDCG@10 ≈ 0.770 with significant improvements over PLAID) and consistently high MRR across test sets, while also maintaining strong performance on DL-HARD. In contrast, RM-style weighting (RM1/RM3) can yield higher top-10 gain on DL’20 (RM1 attains

the best nDCG@10), yet is less consistent across collections. Bo1 is generally weaker, likely because centroid codes may not exhibit the frequency statistics that Bo1 expects (see Section 4.2).

Impact of expansion selection method. For selecting expansion vectors, Table 4 shows that random selection underperforms both MMR and Top-k, highlighting that effective expansion requires principled token choice and redundancy control to improve query representations. Moreover, MMR markedly improves over Top-k selection on DL’19 and DL-HARD, with the largest advantage on DL-HARD (and significant gains over PLAID), indicating that redundancy control and coverage are particularly important for difficult queries. Overall, while Top-k selection is slightly better on DL’20, MMR yields a stronger overall effectiveness profile by avoiding near-duplicate expansion directions and making better use of the limited f_e expansion slots.

6.6 RQ6: Qualitative Analysis

Table 5 illustrates the qualitative behaviour of PLAID-PRF expansions under a leave-one-out (LOO) analysis [43, 46]. For each query, we first record the selected expansion vectors produced by PLAID-PRF, then maintain the aligned WordPiece token IDs for these expansions and look them up and de-tokenise them back to their token forms. To quantify each token’s marginal utility, we perform LOO by removing one expansion vector at a time while keeping all other settings unchanged, and report the resulting change in retrieval effectiveness in terms of MAP.

Overall, the improved queries show that high-contribution expansions are typically domain-indicative tokens that sharpen query intent and improve early ranking. For “what is a active margin”, tokens such as ‘east’, ‘pacific’, ‘continental’, ‘earthquake’, and ‘boundary’ form a coherent geoscience context, which helps disambiguate “active margin” and promotes relevant passages. For “definition of a sigmet”, expansions concentrate on aviation/meteorology vocabulary (e.g., ‘meteorological’, ‘advisory’, ‘safety’), consistent with the intended definition and yielding concentrated positive contributions. In contrast, “tracheids are part of” exhibits a mixed pattern: while several expansions are mildly helpful, a subset of generic or fragmentary tokens (e.g., ‘tr’, ‘linking’, ‘to’, ‘##a’) receives negative LOO contributions, indicating that some expansion vectors can introduce noisy matching signals that distract late-interaction alignment. This contrast motivates token-level or code-level filtering to reduce non-informative expansions and improve robustness.

Table 5: Qualitative examples of PLAID-PRF expansions. Token background colour indicates the leave-one-out marginal contribution on retrieval effectiveness (darker colour denotes higher contribution, red for $\Delta > 0$ and blue for $\Delta < 0$).

Original Query	Expansion Tokens
what is a active margin	east, [CLS], pacific, crash, [SEP], continental, of, break, scar, earthquake, boundary, and, margin, ##ic
definition of a sigmet	area, 3, safety, a, criteria, advisory, to, ##ctive, [CLS], of, meteorological, issued, ##c, (
tracheids are part of	one, system, of, vessel, x, [CLS], maturity, other, ., the, tr, linking, to, ##a

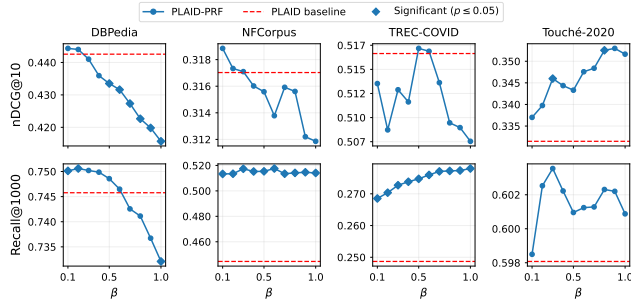


Figure 6: Effect of β of PLAID-PRF across selected BEIR datasets, measured by nDCG@10 & Recall@1000.

6.7 RQ7: Out-of-domain Performance

Experimental Setup. We now examine PLAID-PRF on the out-of-domain retrieval BEIR benchmarks. In particular, as these datasets widely differ, we vary β , which controls the contribution of expansion vectors, separately on each of the selected BEIR datasets, i.e., DBPedia, NFCorpus, TREC-COVID and Touché-2020 (v2).

Results. Figure 6 shows the performance of the various β values for nDCG@10 and Recall@1000. We observe that while for some datasets, like DBPedia and NFCorpus, PLAID-PRF does not benefit precision, and small β values are preferred; TREC-Covid benefits for $0.5 \leq \beta \leq 0.6$; in contrast, Touché-2020 benefits for all values of β . Indeed, Touché-2020 is a counter-argument retrieval dataset, and PRF allows more relevant counter-arguments to be retrieved than the relevance trained-retrieval model. On the other hand, for Recall@1000, we observe that for all datasets performance is markedly enhanced across a wider range of β values than for nDCG@10 (significantly so for 3 datasets).

Overall, in response to RQ7, these results suggest that the proposed feedback method remains effective beyond the training domain and can improve both top-ranked precision and deeper retrieval coverage in zero-shot settings.

7 Conclusions

In this paper, we propose PLAID-PRF, a novel pseudo-relevance feedback method within the quantised late-interaction retrieval paradigm. PLAID-PRF builds the feedback model directly from PLAID/ColBERTv2 centroid codes, enabling lightweight usefulness scoring over the PRF set without operating in the full-precision space. A small set of weighted expansion vectors is then selected

(e.g., via MMR) and appended to the original query for a second-pass PLAID rerun. Experiments on the TREC Deep Learning and DL-HARD datasets show that PLAID-PRF improves nDCG@10 and early precision over PLAID while keeping latency overhead modest, yielding a competitive effectiveness–efficiency operating point among existing strong neural baselines. On out-of-domain BEIR benchmarks, PLAID-PRF also remains competitive and often improves upon PLAID, particularly for Recall. These results indicate that PRF can be made practical for PLAID-style retrieval when feedback modelling and selection are aligned with the index’s compressed structure, opening up the possibility of practical PRF deployments on state-of-the-art PLAID-based search engines.

For future work, a natural direction suggested by our sensitivity analysis is adaptive query expansion for quantised late-interaction retrieval, where the system selects query-specific expansion hyperparameters. For instance, expanding more aggressively for under-specified queries while keeping minimal expansions for precise navigational queries, to maximise potential effective improvements while avoid degradation that can hinder user experience.

Acknowledgements

Xiao Wang acknowledges support from the Fundamental Research Funds for the Central Universities in UIBE (Grant No.24QN01) and from the Humanities and Social Sciences Research Project of the Ministry of Education of China (Grant No.25YJCZH267). The authors are thankful to reviewers for their constructive comments and suggestions, and Zhili Shen for insightful feedback.

References

- [1] Giambattista Amati, Claudio Carpineto, and Giovanni Romano. 2004. Query Difficulty, Robustness, and Selective Application of Query Expansion. In *Advances in Information Retrieval, 26th European Conference on IR Research, ECIR 2004, Sunderland, UK, April 5-7, 2004, Proceedings (Lecture Notes in Computer Science, Vol. 2997)*. Springer, 127–137. doi:10.1007/978-3-540-24752-4_10
- [2] Gianni Amati and C. J. van Rijsbergen. 2002. Probabilistic models of information retrieval based on measuring the divergence from randomness. *ACM Trans. Inf. Syst.* 20, 4 (2002), 357–389. doi:10.1145/582415.582416
- [3] Sanjeev Arora, Yingyu Liang, and Tengyu Ma. 2017. A Simple but Tough-to-Beat Baseline for Sentence Embeddings. In *5th International Conference on Learning Representations, ICLR 2017, Toulon, France, April 24-26, 2017, Conference Track Proceedings*. OpenReview.net. <https://openreview.net/forum?id=SyK00v5xx>
- [4] Jaime G. Carbonell and Jade Goldstein. 2017. The Use of MMR, Diversity-Based Reranking for Reordering Documents and Producing Summaries. *SIGIR Forum* 51, 2 (2017), 209–210. doi:10.1145/3130348.3130369
- [5] Antoine Chaffin. 2025. GTE-ModernColBERT. <https://huggingface.co/lightonai/GTE-ModernColBERT-v1>
- [6] Jianlyu Chen, Shitao Xiao, Peitian Zhang, Kun Luo, Defu Lian, and Zheng Liu. 2024. M3-Embedding: Multi-Linguality, Multi-Functionality, Multi-Granularity Text Embeddings Through Self-Knowledge Distillation. In *Findings of the Association for Computational Linguistics, ACL 2024, Bangkok, Thailand and virtual meeting, August 11-16, 2024. Association for Computational Linguistics*, 2318–2335. doi:10.18653/V1/2024.FINDINGS-ACL.137
- [7] Charles L. A. Clarke, Maheedhar Kolla, Gordon V. Cormack, Olga Vechtomova, Azin Ashkan, Stefan Büttcher, and Ian MacKinnon. 2008. Novelty and diversity in information retrieval evaluation. In *Proceedings of the 31st Annual International ACM SIGIR Conference on Research and Development in Information Retrieval, SIGIR 2008, Singapore, July 20-24, 2008. ACM*, 659–666. doi:10.1145/1390334.1390446
- [8] Nick Craswell, Bhaskar Mitra, Emine Yilmaz, and Daniel Campos. 2020. Overview of the TREC 2020 Deep Learning Track. In *Proceedings of the Twenty-Ninth Text Retrieval Conference, TREC 2020, Virtual Event [Gaithersburg, Maryland, USA], November 16-20, 2020 (NIST Special Publication, Vol. 1266)*. National Institute of Standards and Technology (NIST). https://trec.nist.gov/pubs/trec29/papers/OVERVIEW_DL.pdf
- [9] Nick Craswell, Bhaskar Mitra, Emine Yilmaz, Daniel Campos, and Ellen M. Voorhees. 2020. Overview of the TREC 2019 deep learning track. *CoRR abs/2003.07820* (2020). arXiv:2003.07820 <https://arxiv.org/abs/2003.07820>

- [10] W. Bruce Croft and David J. Harper. 1979. Using Probabilistic Models of Document Retrieval without Relevance Information. *J. Documentation* 35, 4 (1979), 285–295. doi:10.1108/EB026683
- [11] Lukas Galke, Ahmed Saleh, and Ansgar Scherp. 2017. Word Embeddings for Practical Information Retrieval. In *47. Jahrestagung der Gesellschaft für Informatik, Digitale Kulturen, INFORMATIK 2017, Chemnitz, Germany, September 25–29, 2017 (LNI)*, Maximilian Eibl and Martin Gaedke (Eds.). GI, 2155–2167. doi:10.18420/IN2017_215
- [12] Luyu Gao, Zhuyun Dai, Tongfei Chen, Zhen Fan, Benjamin Van Durme, and Jamie Callan. 2021. Complement Lexical Retrieval Model with Semantic Residual Embeddings. In *Advances in Information Retrieval - 43rd European Conference on IR Research, ECIR 2021, Virtual Event, March 28 - April 1, 2021, Proceedings, Part I (Lecture Notes in Computer Science, Vol. 12656)*. Springer, 146–160. doi:10.1007/978-3-030-72113-8_10
- [13] Luyu Gao, Xueguang Ma, Jimmy Lin, and Jamie Callan. 2023. Precise Zero-Shot Dense Retrieval without Relevance Labels. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), ACL 2023, Toronto, Canada, July 9–14, 2023*. Association for Computational Linguistics, 1762–1777. doi:10.18653/V1/2023.ACL-LONG.99
- [14] Fatima Haouari, Marwa Essam, and Tamer Elsayed. 2020. bigIR at TREC 2020: Simple but Deep Retrieval of Passages and Documents. In *Proceedings of the Twenty-Ninth Text REtrieval Conference, TREC 2020, Virtual Event [Gaithersburg, Maryland, USA], November 16–20, 2020 (NIST Special Publication, Vol. 1266)*. National Institute of Standards and Technology (NIST). <https://trec.nist.gov/pubs/trec29/papers/QU.DL.pdf>
- [15] Nasreen Abdul Jaleel, James Allan, W. Bruce Croft, Fernando Diaz, Leah S. Larkey, Xiaoyan Li, Mark D. Smucker, and Courtney Wade. 2004. UMass at TREC 2004: Novelty and HARD. In *Proceedings of the Thirteenth Text REtrieval Conference, TREC 2004, Gaithersburg, Maryland, USA, November 16–19, 2004 (NIST Special Publication, Vol. 500-261)*. National Institute of Standards and Technology (NIST). <http://trec.nist.gov/pubs/trec13/papers/umass.novelty.hard.pdf>
- [16] Jeff Johnson, Matthijs Douze, and Hervé Jégou. 2021. Billion-Scale Similarity Search with GPUs. *IEEE Trans. Big Data* 7, 3 (2021), 535–547. doi:10.1109/TBDDATA.2019.2921572
- [17] Ehsan Kamalloo, Nandan Thakur, Carlos Lassance, Xueguang Ma, Jheng-Hong Yang, and Jimmy Lin. 2023. Resources for Brewing BEIR: Reproducible Reference Models and an Official Leaderboard. *CoRR abs/2306.07471* (2023). arXiv:2306.07471 doi:10.48550/ARXIV.2306.07471
- [18] Omar Khattab and Matei Zaharia. 2020. ColBERT: Efficient and Effective Passage Search via Contextualized Late Interaction over BERT. In *Proceedings of the 43rd International ACM SIGIR conference on research and development in Information Retrieval, SIGIR 2020, Virtual Event, China, July 25–30, 2020*. ACM, 39–48. doi:10.1145/3397271.3401075
- [19] Hrishikesh Kulkarni, Sean MacAvaney, Nazli Goharian, and Ophir Frieder. 2023. Lexically-Accelerated Dense Retrieval. In *Proceedings of the 46th International ACM SIGIR Conference on Research and Development in Information Retrieval, SIGIR 2023, Taipei, Taiwan, July 23–27, 2023*. ACM, 152–162. doi:10.1145/3539618.3591715
- [20] Victor Lavrenko and W. Bruce Croft. 2017. Relevance-Based Language Models. *SIGIR Forum* 51, 2 (2017), 260–267. doi:10.1145/3130348.3130376
- [21] Jinhyuk Lee, Zhuyun Dai, Sai Meher Karthik Duddu, Tao Lei, Iftekhar Naim, Ming-Wei Chang, and Vincent Zhao. 2023. Rethinking the Role of Token Retrieval in Multi-Vector Retrieval. In *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023*. http://papers.nips.cc/paper_files/paper/2023/hash/31d997278ee9069d6721bc194174bb4c-Abstract-Conference.html
- [22] Hang Li, Ahmed Mourad, Shengyao Zhuang, Bevan Koopman, and Guido Zuccon. 2021. Pseudo Relevance Feedback with Deep Language Models and Dense Retrievers: Successes and Pitfalls. *CoRR abs/2108.11044* (2021). arXiv:2108.11044 <https://arxiv.org/abs/2108.11044>
- [23] Hang Li, Xiao Wang, Bevan Koopman, and Guido Zuccon. 2025. Pseudo Relevance Feedback is Enough to Close the Gap Between Small and Large Dense Retrieval Models. *arXiv preprint arXiv:2503.14887* (2025).
- [24] Hang Li, Shengyao Zhuang, Bevan Koopman, and Guido Zuccon. 2025. LLM-VPRF: Large Language Model Based Vector Pseudo Relevance Feedback. *CoRR abs/2504.01448* (2025). arXiv:2504.01448 doi:10.48550/ARXIV.2504.01448
- [25] Xueguang Ma, Liang Wang, Nan Yang, Furu Wei, and Jimmy Lin. 2024. Fine-Tuning LLaMA for Multi-Stage Text Retrieval. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval, SIGIR 2024, Washington DC, USA, July 14–18, 2024*. ACM, 2421–2425. doi:10.1145/3626772.3657951
- [26] Sean MacAvaney, Antonio Mallia, and Nicola Tonellotto. 2025. Efficient Constant-Space Multi-vector Retrieval. In *Advances in Information Retrieval - 47th European Conference on Information Retrieval, ECIR 2025, Lucca, Italy, April 6–10, 2025, Proceedings, Part III (Lecture Notes in Computer Science, Vol. 15574)*. Springer, 237–245. doi:10.1007/978-3-031-88714-7_22
- [27] Sean MacAvaney and Nicola Tonellotto. 2024. A Reproducibility Study of PLAID. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval, SIGIR 2024, Washington DC, USA, July 14–18, 2024*. ACM, 1411–1419. doi:10.1145/3626772.3657856
- [28] Sean MacAvaney, Nicola Tonellotto, and Craig Macdonald. 2022. Adaptive Re-Ranking with a Corpus Graph. In *Proceedings of the 31st ACM International Conference on Information and Knowledge Management, Atlanta, GA, USA, October 17–21, 2022*. ACM, 1491–1500. doi:10.1145/3511808.3557231
- [29] Craig Macdonald, Nicola Tonellotto, and Iadh Ounis. 2021. PyTerrier: Declarative Experimentation in Python from BM25 to Dense Retrieval. In *CIKM '21: The 30th ACM International Conference on Information and Knowledge Management, Virtual Event, Queensland, Australia, November 1 - 5, 2021*. ACM, 4526–4533. doi:10.1145/3459637.3482013
- [30] Iain Mackie, Shubham Chatterjee, and Jeffrey Dalton. 2023. Generative Relevance Feedback with Large Language Models. In *Proceedings of the 46th International ACM SIGIR Conference on Research and Development in Information Retrieval, SIGIR 2023, Taipei, Taiwan, July 23–27, 2023*. ACM, 2026–2031. doi:10.1145/3539618.3591992
- [31] Iain Mackie, Jeffrey Dalton, and Andrew Yates. 2021. How Deep is your Learning: the DL-HARD Annotated Deep Learning Dataset. In *SIGIR '21: The 44th International ACM SIGIR Conference on Research and Development in Information Retrieval, Virtual Event, Canada, July 11–15, 2021*. ACM, 2335–2341. doi:10.1145/3404835.3463262
- [32] Yury A. Malkov and Dmitry A. Yashunin. 2016. Efficient and robust approximate nearest neighbor search using Hierarchical Navigable Small World graphs. *CoRR abs/1603.09320* (2016). arXiv:1603.09320 <http://arxiv.org/abs/1603.09320>
- [33] Christopher D Manning. 2009. *An introduction to information retrieval*. Cambridge university press.
- [34] Shahrzad Naseri, Jeff Dalton, Andrew Yates, and James Allan. 2021. CEQE: Contextualized Embeddings for Query Expansion. In *Advances in Information Retrieval - 43rd European Conference on IR Research, ECIR 2021, Virtual Event, March 28 - April 1, 2021, Proceedings, Part I (Lecture Notes in Computer Science, Vol. 12656)*. Springer, 467–482. doi:10.1007/978-3-030-72113-8_31
- [35] Tri Nguyen, Mir Rosenberg, Xia Song, Jianfeng Gao, Saurabh Tiwary, Rangan Majumder, and Li Deng. 2016. MS MARCO: A Human Generated Machine Reading Comprehension Dataset. In *Proceedings of the Workshop on Cognitive Computation: Integrating neural and symbolic approaches 2016 co-located with the 30th Annual Conference on Neural Information Processing Systems (NIPS 2016), Barcelona, Spain, December 9, 2016 (CEUR Workshop Proceedings, Vol. 1773)*. CEUR-WS.org. https://ceur-ws.org/Vol-1773/CoCoNIPS_2016_paper9.pdf
- [36] Mandeep Rathee, Sean MacAvaney, and Avishek Anand. 2025. Qam: Adaptive Retrieval through Query Affinity Modelling. In *Proceedings of the Eighteenth ACM International Conference on Web Search and Data Mining, WSDM 2025, Hannover, Germany, March 10–14, 2025*. ACM, 954–962. doi:10.1145/3701551.3703584
- [37] Mandeep Rathee, Venkatesh V, Sean MacAvaney, and Avishek Anand. 2025. Breaking the Lens of the Telescope: Online Relevance Estimation over Large Retrieval Sets. In *Proceedings of the 48th International ACM SIGIR Conference on Research and Development in Information Retrieval, SIGIR 2025, Padua, Italy, July 13–18, 2025*. ACM, 2287–2297. doi:10.1145/3726302.3729910
- [38] Joseph Rocchio. 1971. Relevance feedback in information retrieval. *The Smart Retrieval System-experiments in Automatic Document Processing* (1971), 313–323.
- [39] Keshav Santhanam, Omar Khattab, Christopher Potts, and Matei Zaharia. 2022. PLAID: An Efficient Engine for Late Interaction Retrieval. In *Proceedings of the 31st ACM International Conference on Information and Knowledge Management, Atlanta, GA, USA, October 17–21, 2022*. ACM, 1747–1756. doi:10.1145/3511808.3557325
- [40] Keshav Santhanam, Omar Khattab, Jon Saad-Falcon, Christopher Potts, and Matei Zaharia. 2022. ColBERTv2: Effective and Efficient Retrieval via Lightweight Late Interaction. In *Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, NAACL 2022, Seattle, WA, United States, July 10–15, 2022*. Association for Computational Linguistics, 3715–3734. doi:10.18653/V1/2022.NAAACL-MAIN.272
- [41] Rodrygo L. T. Santos, Craig Macdonald, and Iadh Ounis. 2010. Exploiting query reformulations for web search result diversification. In *Proceedings of the 19th International Conference on World Wide Web, WWW 2010, Raleigh, North Carolina, USA, April 26–30, 2010*. ACM, 881–890. doi:10.1145/1772690.1772780
- [42] Nandan Thakur, Nils Reimers, Andreas Rücklé, Abhishek Srivastava, and Iryna Gurevych. 2021. BEIR: A Heterogeneous Benchmark for Zero-shot Evaluation of Information Retrieval Models. *CoRR abs/2104.08663* (2021). arXiv:2104.08663 <https://arxiv.org/abs/2104.08663>
- [43] Xiao Wang, Sean MacAvaney, Craig Macdonald, and Iadh Ounis. 2023. Effective Contrastive Weighting for Dense Query Expansion. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), ACL 2023, Toronto, Canada, July 9–14, 2023*. Association for Computational Linguistics, 12688–12704. doi:10.18653/V1/2023.ACL-LONG.710
- [44] Xiao Wang, Sean MacAvaney, Craig Macdonald, and Iadh Ounis. 2023. Generative Query Reformulation for Effective Adhoc Retrieval. In *Proceedings of Workshop on GenIR*.
- [45] Xiao Wang, Craig Macdonald, and Iadh Ounis. 2022. Improving zero-shot retrieval using dense external expansion. *Inf. Process. Manag.* 59, 5 (2022), 103026. doi:10.

- 1016/J.IPM.2022.103026
- [46] Xiao Wang, Craig MacDonald, Nicola Tonellotto, and Iadh Ounis. 2023. ColBERT-PRF: Semantic Pseudo-Relevance Feedback for Dense Passage and Document Retrieval. *ACM Trans. Web* 17, 1 (2023), 3:1–3:39. doi:10.1145/3572405
 - [47] Xiao Wang, Craig Macdonald, Nicola Tonellotto, and Iadh Ounis. 2023. Reproducibility, Replicability, and Insights into Dense Multi-Representation Retrieval Models: from ColBERT to Col★. In *Proceedings of SIGIR*.
 - [48] Lee Xiong, Chenyan Xiong, Ye Li, Kwok-Fung Tang, Jialin Liu, Paul N. Bennett, Junaid Ahmed, and Arnold Overwijk. 2021. Approximate Nearest Neighbor Negative Contrastive Learning for Dense Text Retrieval. In *9th International Conference on Learning Representations, ICLR 2021, Virtual Event, Austria, May 3-7, 2021*. OpenReview.net. <https://openreview.net/forum?id=zeFrFgyZln>
 - [49] HongChien Yu, Chenyan Xiong, and Jamie Callan. 2021. Improving Query Representations for Dense Retrieval with Pseudo Relevance Feedback. In *CIKM '21: The 30th ACM International Conference on Information and Knowledge Management, Virtual Event, Queensland, Australia, November 1 - 5, 2021*. ACM, 3592–3596. doi:10.1145/3459637.3482124
 - [50] Hamed Zamani and W. Bruce Croft. 2016. Embedding-based Query Language Models. In *Proceedings of the 2016 ACM on International Conference on the Theory of Information Retrieval, ICTIR 2016, Newark, DE, USA, September 12- 6, 2016*. ACM, 147–156. doi:10.1145/2970398.2970405
 - [51] Zhi Zheng, Kai Hui, Ben He, Xianpei Han, Le Sun, and Andrew Yates. 2020. BERT-QE: Contextualized Query Expansion for Document Re-ranking. In *Findings of the Association for Computational Linguistics: EMNLP 2020, Online Event, 16-20 November 2020 (Findings of ACL, Vol. EMNLP 2020)*. Association for Computational Linguistics, 4718–4728. doi:10.18653/V1/2020.FINDINGS-EMNLP.424
 - [52] Shengyao Zhuang, Xueguang Ma, Bevan Koopman, Jimmy Lin, and Guido Zuccon. 2024. PromptReps: Prompting Large Language Models to Generate Dense and Sparse Representations for Zero-Shot Document Retrieval. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing, EMNLP 2024, Miami, FL, USA, November 12-16, 2024*. Association for Computational Linguistics, 4375–4391. doi:10.18653/V1/2024.EMNLP-MAIN.250